

Executor Framework in Java

Java allows us to create threads directly using the `Thread` class. That approach is useful for understanding the basics of multithreading, but it does not scale well when an application needs to execute many independent tasks.

The Executor Framework provides a higher-level and more controlled way to manage task execution.

1. Why Creating Threads Manually Is Not Ideal

A common manual approach looks like this:

```
Thread thread = new Thread(() -> processOrder());
thread.start();
```

This works, but the code is responsible for both:

- defining the task
- deciding how the task will be executed
- creating the thread
- managing the thread's lifecycle

For a small demonstration, this is acceptable. For a production application, repeatedly creating threads introduces several problems.

1.1 Threads Consume Resources

A Java platform thread is generally backed by an operating-system thread.

Each thread requires resources such as:

- native stack memory
- JVM and operating-system bookkeeping
- CPU scheduling
- context switching

The exact stack size depends on the JVM, operating system and configuration, but creating a very large number of threads can consume substantial native memory.

1.2 Context-Switching Overhead

When several runnable threads compete for a limited number of CPU cores, the operating system repeatedly switches between them.

A context switch may involve:

- saving the state of the current thread
- restoring the state of another thread
- disrupting CPU-cache locality
- spending CPU time on scheduling rather than useful work

Too many active threads can therefore reduce performance instead of improving it.

1.3 Thread Creation Has a Cost

Creating and destroying an operating-system thread is more expensive than executing a lightweight Java method call.

When every task creates a separate thread, the application repeatedly pays this cost.

1.4 No Thread Reuse

A manually created thread normally executes its task and then terminates.

```
Create thread → execute one task → terminate thread
```

The next task requires another thread to be created.

1.5 No Natural Upper Bound

Nothing in the following code prevents an application from creating thousands of threads:

```
for (Request request : requests) {  
    new Thread(() -> process(request)).start();  
}
```

If requests arrive faster than the system can process them, thread creation may continue until the application runs out of memory or becomes unresponsive.

1.6 Difficult Lifecycle Management

After calling `start()`, the application must manually coordinate thread completion, interruption and shutdown.

Methods such as `join()` and `interrupt()` help, but they do not provide a complete task-management system.

Mental model: Creating one thread for every task is like hiring a new chef for every order and dismissing the chef after preparing one dish.

2. What Is the Executor Framework?

The Executor Framework is Java's built-in abstraction for submitting tasks without directly managing the threads that execute them.

Its central idea is:

Separate task submission from task execution.

Without an Executor

```
Thread thread = new Thread(() -> processOrder(order));  
thread.start();
```

The code decides:

- what should run
- which thread should run it
- when that thread should be created

With an Executor

```
executor.execute(() -> processOrder(order));
```

The code only submits the task.

The executor decides whether the task should:

- run immediately
- wait in a queue
- run on an existing worker thread
- cause another worker thread to be created
- be rejected because the system is overloaded

3. Core Architecture

```
Task
(Runnable or Callable)
    ↓
Executor / ExecutorService
    ↓
Task Queue
    ↓
Worker Threads
    ↓
Task Execution
```

The framework normally works with two separate concepts:

Concept	Responsibility
Task	Describes the work that must be performed
Executor	Decides how and when the task will run

A **thread** is a low-level execution unit.

An **executor** is a higher-level abstraction that manages task execution using threads.

4. First Example: Fixed Thread Pool

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ExecutorDemo {

    public static void main(String[] args) {

        ExecutorService executor =
            Executors.newFixedThreadPool(2);

        for (int i = 1; i <= 5; i++) {
            int taskId = i;

            executor.execute(() -> {
                System.out.println(
                    "Task " + taskId +
                    " executed by " +
                    Thread.currentThread().getName()
                );
            });
        }

        executor.shutdown();
    }
}
```

What Happens Internally?

1. A pool capable of using two worker threads is created.
2. Five tasks are submitted.
3. At most two tasks execute at the same time.
4. Remaining tasks wait in the work queue.

5. After completing one task, a worker thread picks another task.
6. The same threads are reused.
7. `shutdown()` prevents new submissions and allows accepted tasks to finish.

```
Two worker threads
+
Five independent tasks
=
Threads reused across multiple tasks
```

The output order is not guaranteed because tasks run concurrently.

5. The Task Queue

A thread pool usually contains a queue for tasks that cannot run immediately.

```
Submitted task
↓
Available worker?
├─ Yes → execute task
└─ No  → place task in queue
```

This prevents the application from creating a new thread for every task.

However, a queue does not automatically make an executor safe. An unbounded queue can continue growing if tasks arrive faster than they are completed.

6. Common Use Cases

The Executor Framework is commonly used for:

- processing independent requests
- background jobs
- batch processing
- asynchronous service calls
- database or file operations

- parallel computations
- scheduled and periodic tasks
- message and event processing

The executor should be configured according to the workload. CPU-intensive work and blocking I/O work usually require different pool-sizing strategies.

Executor Framework Hierarchy

```
Executor
  ↓
ExecutorService
  ↓
AbstractExecutorService
  ↓
ThreadPoolExecutor
```

For scheduled execution:

```
ExecutorService
  ↓
ScheduledExecutorService
  ↓
ScheduledThreadPoolExecutor
```

7. The **Executor** Interface

Executor is the smallest abstraction in the framework.

```
public interface Executor {
    void execute(Runnable command);
}
```

It has only one method:

```
void execute(Runnable command);
```

- `Runnable` describes the task.
- `execute()` requests that the task be executed.
- The interface does not define exactly where, when or on which thread it must run.

Different Executor Implementations Can Behave Differently

Run in the Current Thread

```
class DirectExecutor implements Executor {  
  
    @Override  
    public void execute(Runnable command) {  
        command.run();  
    }  
}
```

This is a valid executor even though it does not create another thread.

Create a New Thread for Every Task

```
class NewThreadExecutor implements Executor {  
  
    @Override  
    public void execute(Runnable command) {  
        new Thread(command).start();  
    }  
}
```

Use a Thread Pool

```
Executor executor = Executors.newFixedThreadPool(4);
executor.execute(task);
```

A thread pool is one possible implementation of the `Executor` abstraction.

8. The `ExecutorService` Interface

`ExecutorService` extends `Executor` and adds features required for practical task management.

```
ExecutorService
=
task execution
+
task results
+
task cancellation
+
bulk task operations
+
lifecycle management
```

Important methods include:

```
execute(...)
submit(...)
invokeAll(...)
invokeAny(...)
shutdown()
shutdownNow()
awaitTermination(...)
```

Executing and Submitting Tasks

9. `execute()`

```
executor.execute(task);
```

`execute()`:

- accepts a `Runnable`
- does not return a result
- does not return a `Future`
- is suitable for fire-and-forget tasks

Example:

```
executor.execute(() -> {
    System.out.println("Sending notification");
});
```

Because no result object is returned, the caller cannot directly check completion or cancel this particular task through a `Future`.

10. `submit()`

`submit()` returns a `Future` representing the submitted task.

Submit a `Callable`

```
Future<Integer> future = executor.submit(() -> 10 + 20);
```

Submit a `Runnable`

```
Future<?> future = executor.submit(() -> {
    System.out.println("Task completed");
});
```

For a successfully completed `Runnable`, `future.get()` returns `null`.

Submit a `Runnable` With a Predefined Result

```
Future<String> future = executor.submit(
    () -> System.out.println("Saving data"),
    "SUCCESS"
);
```

After successful completion:

```
System.out.println(future.get()); // SUCCESS
```

execute() vs submit()

Feature	execute()	submit()
Accepted task	Runnable	Runnable or Callable
Return value	Nothing	Future
Per-task cancellation through Future	No	Yes
Completion tracking	No direct handle	Available through Future
Task exception	Reaches the worker thread's uncaught-exception mechanism	Stored in the Future
Typical use	Fire-and-forget task	Result, cancellation or completion tracking

Runnable and Callable

11. Runnable

```
@FunctionalInterface
public interface Runnable {
    void run();
}
```

Characteristics:

- does not return a value
- cannot declare checked exceptions in `run()`
- can be used with both `execute()` and `submit()`

Example:

```
Runnable task = () -> {
    System.out.println("Processing data");
};
```

12. Callable

```
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}
```

Characteristics:

- returns a value
- can throw checked exceptions
- is submitted using `submit()`, `invokeAll()` or `invokeAny()`

Example:

```
Callable<Integer> task = () -> {
    Thread.sleep(500);
    return 100;
};
```

Runnable vs Callable

Feature	Runnable	Callable<V>
Method	<code>run()</code>	<code>call()</code>

Feature	Runnable	Callable<V>
Return value	No	Yes
Checked exceptions	Cannot declare them	Can declare them
Can be passed to <code>execute()</code>	Yes	No
Can be passed to <code>submit()</code>	Yes	Yes

Future

13. What Is a **Future** ?

A **Future** is a handle to the result or completion state of an asynchronous task.

```
Task submitted now
    ↓
Task may finish later
    ↓
Future represents its pending result
```

Example:

```
ExecutorService executor =
    Executors.newFixedThreadPool(2);

Future<Integer> future = executor.submit(() -> {
    Thread.sleep(1000);
    return 10 + 20;
});

System.out.println("Main thread can do other work");

Integer result = future.get();

System.out.println("Result: " + result);
```

```
executor.shutdown();
```

Important **Future** Methods

```
future.get();  
future.get(timeout, unit);  
future.isDone();  
future.isCancelled();  
future.cancel(true);
```

Method	Purpose
<code>get()</code>	Waits for completion and returns the result
<code>get(timeout, unit)</code>	Waits only for the specified duration
<code>isDone()</code>	Checks whether the task has completed
<code>isCancelled()</code>	Checks whether the task was cancelled
<code>cancel(true)</code>	Requests cancellation and may interrupt the running task

14. **Future.get()** Is Blocking

```
Integer result = future.get();
```

If the result is ready, it is returned immediately.

If the result is not ready, the calling thread waits.

```
Asynchronous execution  
+  
Blocking retrieval when get() is called
```

Submitting a task is asynchronous. Calling `get()` immediately may make the calling code wait and therefore behave synchronously at that point.

15. Timed `get()`

A timed call prevents the caller from waiting forever.

```
try {
    Integer result =
        future.get(2, TimeUnit.SECONDS);

    System.out.println(result);
} catch (TimeoutException exception) {
    System.out.println("Task did not finish in time");
}
```

16. Cancelling a Task

```
boolean cancelled = future.cancel(true);
```

Passing `true` means that, if the task is already running, the executor may interrupt its worker thread.

Cancellation is cooperative. The task must respond correctly to interruption.

```
Callable<Void> task = () -> {
    while (!Thread.currentThread().isInterrupted()) {
        // Perform work
    }

    return null;
};
```

Calling `cancel(true)` does not forcibly terminate arbitrary Java code.

Exception Handling

17. Exception With `execute()`

```
executor.execute(() -> {  
    int result = 10 / 0;  
});
```

The exception is not returned to the submitting thread.

It becomes an uncaught exception on the worker thread and is normally reported through that thread's `UncaughtExceptionHandler`. In a thread pool, the failed worker may terminate and the pool may create a replacement worker when required.

Use task-level exception handling when the application needs controlled recovery:

```
executor.execute(() -> {  
    try {  
        int result = 10 / 0;  
    } catch (ArithmeticException exception) {  
        System.out.println(  
            "Task failed: " + exception.getMessage()  
        );  
    }  
});
```

18. Exception With `submit()`

```
Future<Integer> future = executor.submit(() -> {  
    return 10 / 0;  
});
```

The exception is captured by the task's `Future`.

```
try {  
    Integer result = future.get();  
} catch (ExecutionException exception) {
```

```
Throwable actualCause = exception.getCause();

System.out.println(
    "Task failed: " + actualCause.getMessage()
);
}
```

`future.get()` throws `ExecutionException`, and the original task exception is available through:

```
exception.getCause();
```

A submitted task's exception can be easy to miss if its `Future` is ignored and `get()` is never called.

Managing Executor Lifecycle

19. Why Shutdown Is Necessary

Worker threads are normally non-daemon threads. If an executor remains active, it may keep the JVM running even after `main()` completes.

Therefore, an executor that is no longer needed should be shut down.

20. `shutdown()`

```
executor.shutdown();
```

`shutdown()`:

- stops accepting new tasks
- allows previously submitted tasks to complete
- initiates an orderly shutdown
- returns immediately without waiting for termination

Submitting a new task after shutdown causes `RejectedExecutionException`.

21. `shutdownNow()`

```
List<Runnable> pendingTasks =  
    executor.shutdownNow();
```

`shutdownNow()`:

- stops accepting new tasks
- removes tasks that have not started
- returns the pending tasks
- attempts to interrupt actively running tasks

It does not guarantee that running tasks will stop immediately. Tasks must cooperate with interruption.

22. `awaitTermination()`

```
boolean terminated =  
    executor.awaitTermination(  
        10,  
        TimeUnit.SECONDS  
    );
```

This method waits for the executor to terminate for up to the specified duration.

It should normally be called after `shutdown()` or `shutdownNow()`.

Graceful Shutdown Pattern

```
executor.shutdown();  
  
try {  
    if (!executor.awaitTermination(  
        10,  
        TimeUnit.SECONDS  
    )) {  
        // If the executor did not terminate within the specified  
        // duration, it is likely that there are tasks that are  
        // not cooperating with interruption. In this case, you  
        // may want to call shutdownNow() to attempt to stop  
        // the tasks as quickly as possible.    }  
}
```

```

        10,
        TimeUnit.SECONDS
    )) {
        executor.shutdownNow();

        if (!executor.awaitTermination(
            10,
            TimeUnit.SECONDS
        )) {
            System.out.println(
                "Executor did not terminate"
            );
        }
    }
} catch (InterruptedException exception) {
    executor.shutdownNow();
    Thread.currentThread().interrupt();
}

```

Working With Multiple Tasks

23. `invokeAll()`

`invokeAll()` executes a collection of `Callable` tasks and waits until all of them complete.

```

ExecutorService executor =
    Executors.newFixedThreadPool(3);

List<Callable<Integer>> tasks = List.of(
    () -> 10,
    () -> 20,
    () -> 30
);

```

```
List<Future<Integer>> futures =  
    executor.invokeAll(tasks);  
  
for (Future<Integer> future : futures) {  
    System.out.println(future.get());  
}  
  
executor.shutdown();
```

Important points:

- the method blocks until all tasks complete
- it returns one `Future` for each task
- returned futures follow the same order as the input task list
- a completed task may have completed successfully or exceptionally

24. `invokeAny()`

`invokeAny()` executes multiple `Callable` tasks and returns the result of one that completes successfully.

```
Integer result = executor.invokeAny(tasks);
```

Once a successful result is available, unfinished tasks are cancelled.

This is useful when the same result can be obtained from multiple independent sources.

Example:

```
List<Callable<String>> servers = List.of(  
    () -> callServerA(),  
    () -> callServerB(),  
    () -> callServerC()  
);
```

```
String response = executor.invokeAny(servers);
```

It returns the first **successful** result, not necessarily the first task that terminates. A task that fails does not provide a successful result.

ThreadPoolExecutor

25. Why a Thread Pool Needs Rules

A complete thread pool requires:

1. a controlled number of worker threads
2. a queue for waiting tasks
3. rules for creating additional threads
4. rules for removing idle threads
5. a policy for handling overload

`ThreadPoolExecutor` provides this configurable execution engine.

```
Incoming tasks
  ↓
ThreadPoolExecutor
  ↓
Workers + Work Queue
  ↓
Task execution
```

26. Executors vs ThreadPoolExecutor

```
ExecutorService executor =
    Executors.newFixedThreadPool(3);
```

`Executors` is a utility class containing factory methods.

`ThreadPoolExecutor` is the concrete class that performs the actual thread-pool management.

```
Executors
=
convenient factory methods

ThreadPoolExecutor
=
configurable execution engine
```

Factory methods are convenient, but direct construction is useful when the application needs explicit control over queue capacity, maximum threads, thread naming and rejection behavior.

27. Important Constructor Parameters

A commonly used constructor is:

```
ThreadPoolExecutor executor =
    new ThreadPoolExecutor(
        corePoolSize,
        maximumPoolSize,
        keepAliveTime,
        timeUnit,
        workQueue,
        threadFactory,
        rejectionHandler
    );
```

The central configuration values are:

Parameter	Meaning
<code>corePoolSize</code>	Target number of core worker threads
<code>maximumPoolSize</code>	Maximum number of worker threads allowed
<code>keepAliveTime</code>	Maximum idle time for excess threads

Parameter	Meaning
<code>TimeUnit</code>	Unit of <code>keepAliveTime</code>
<code>workQueue</code>	Holds tasks waiting to execute
<code>ThreadFactory</code>	Controls how worker threads are created
<code>RejectedExecutionHandler</code>	Handles tasks that cannot be accepted

28. `corePoolSize`

```
corePoolSize = 2;
```

When a new task arrives and fewer than two workers exist, the pool normally creates another worker, even if another worker is currently idle.

An important detail:

Core threads are not necessarily created when the executor object is constructed.

By default, core workers are started as tasks arrive.

They can be created in advance using:

```
executor.prestartCoreThread();
executor.prestartAllCoreThreads();
```

Core threads are normally retained even when idle, unless core-thread timeout is explicitly enabled.

29. `maximumPoolSize`

```
maximumPoolSize = 5;
```

This is the maximum number of worker threads the pool may contain.

However, the pool does not automatically grow to this number merely because tasks are waiting.

New threads beyond `corePoolSize` are created only when:

- all core workers are occupied
- the work queue cannot accept another task
- the current worker count is below `maximumPoolSize`

With an unbounded queue, the queue never becomes full, so the pool normally does not grow beyond `corePoolSize`.

30. `workQueue`

The work queue stores submitted tasks that are waiting for a worker.

```
BlockingQueue<Runnable> queue =  
    new ArrayBlockingQueue<>(100);
```

Queue choice directly affects:

- memory usage
- latency
- maximum concurrency
- overload behavior
- whether `maximumPoolSize` is used

31. `keepAliveTime`

```
keepAliveTime = 30;  
TimeUnit unit = TimeUnit.SECONDS;
```

When the number of workers is greater than `corePoolSize`, an excess worker is removed after remaining idle for the keep-alive duration.

Core workers → normally retained

Extra workers → removed after idle timeout

The same timeout can also be applied to core threads:

```
executor.allowCoreThreadTimeout(true);
```

ThreadPoolExecutor Decision Flow

32. What Happens When a Task Is Submitted?

For a typical `ThreadPoolExecutor`, the simplified decision process is:

1. Worker count below `corePoolSize`?
Yes → create worker and run task
2. Otherwise, can the queue accept the task?
Yes → enqueue task
3. Otherwise, worker count below `maximumPoolSize`?
Yes → create extra worker and run task
4. Otherwise
Reject task

The order is important:

The executor fills core threads first, then fills the queue, and only then creates threads beyond the core size.

33. Example Flow

Configuration:

```
corePoolSize = 2;  
maximumPoolSize = 4;  
queueCapacity = 2;
```

Assume every submitted task remains busy long enough for all tasks to arrive.

Task	Result
Task 1	Creates worker T1
Task 2	Creates worker T2
Task 3	Added to queue
Task 4	Added to queue
Task 5	Queue is full, so worker T3 is created
Task 6	Queue is full, so worker T4 is created
Task 7	Maximum workers and queue are full, so task is rejected

State before any task completes:

```
Workers: T1, T2, T3, T4
Queue:   Task 3, Task 4
```

34. Custom `ThreadPoolExecutor` Example

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public class CustomThreadPoolDemo {

    public static void main(String[] args) {

        ThreadPoolExecutor executor =
            new ThreadPoolExecutor(
                2,
                4,
                10,
                TimeUnit.SECONDS,
                new ArrayBlockingQueue<>(2)
```

```

    );

    for (int i = 1; i <= 6; i++) {
        int taskId = i;

        executor.execute(() -> {
            System.out.println(
                "Task " + taskId +
                " executed by " +
                Thread.currentThread().getName()
            );

            try {
                Thread.sleep(2000);
            } catch (InterruptedException exception) {
                Thread.currentThread().interrupt();
            }
        });
    }

    executor.shutdown();
}

```

This executor provides:

- two core workers
- up to four workers under load
- a queue that can hold two waiting tasks
- ten-second keep-alive time for excess workers
- the default rejection policy

35. Worker Thread Lifecycle

```

Create worker
  ↓
Execute task
  ↓
Take another task from queue
  ↓
Become idle
  ↓
Reuse or terminate according to pool rules

```

A worker is not destroyed after every task. Reuse is one of the main benefits of a thread pool.

Work Queue Types

36. **ArrayBlockingQueue**

```
new ArrayBlockingQueue<>(100);
```

Characteristics:

- bounded capacity
- array-backed
- capacity is fixed at construction
- enables predictable overload handling
- allows the pool to grow beyond its core size after the queue becomes full

A bounded queue is often preferable when the system must protect memory and apply backpressure.

37. **LinkedBlockingQueue**

```
new LinkedBlockingQueue<>();
```

Characteristics:

- optionally bounded
- effectively unbounded when no capacity is supplied
- commonly used by fixed thread pools
- can accumulate a very large number of pending tasks

Bounded form:

```
new LinkedBlockingQueue<>(1000);
```

An unbounded queue can hide overload temporarily while steadily consuming memory.

38. SynchronousQueue

```
new SynchronousQueue<>();
```

A `SynchronousQueue` has no internal storage capacity.

Each submitted task must be handed directly to a worker.

It is used by cached thread pools.

```
Task arrives
  ↓
Idle worker available?
  ├── Yes → direct handoff
  └── No  → create another worker, if allowed
```

Executor Factory Methods

39. Fixed Thread Pool

```
ExecutorService executor =
    Executors.newFixedThreadPool(3);
```

Conceptually:

```
corePoolSize = 3
maximumPoolSize = 3
workQueue = unbounded LinkedBlockingQueue
```

Characteristics:

- fixed number of worker threads
- tasks beyond current capacity wait in the queue
- threads are reused
- preserves no guaranteed completion order
- the unbounded queue can grow excessively during sustained overload

Suitable for:

- controlled concurrency
- workloads where the task-arrival rate is reasonably bounded
- cases where a fixed number of concurrent operations is required

40. Cached Thread Pool

```
ExecutorService executor =
    Executors.newCachedThreadPool();
```

Conceptually:

```
corePoolSize = 0
maximumPoolSize = very large
workQueue = SynchronousQueue
keepAliveTime = 60 seconds
```

Characteristics:

- creates threads on demand

- reuses idle workers when available
- idle workers are removed after the keep-alive period
- uses direct handoff rather than storing tasks in a normal queue
- can create a very large number of threads

Suitable for:

- many short-lived asynchronous tasks
- workloads with controlled submission rates

Risk:

A cached thread pool can create too many platform threads when long-running or blocking tasks arrive rapidly.

41. Single-Thread Executor

```
ExecutorService executor =
    Executors.newSingleThreadExecutor();
```

Characteristics:

- one worker thread
- tasks execute sequentially
- tasks wait in an internal queue
- a failed worker is replaced when necessary
- thread reuse is automatic

Task 1 → Task 2 → Task 3

Suitable for:

- ordered event processing
- serial file writing

- single-consumer workflows
- avoiding concurrent access to a resource through confinement

Important limitation:

- its internal queue is unbounded
- a slow worker can cause a large backlog

A single-thread executor guarantees sequential execution, but it does not make unrelated shared state automatically thread-safe if other threads access that state directly.

Scheduled Execution

42. `ScheduledExecutorService`

```
ScheduledExecutorService scheduler =  
    Executors.newScheduledThreadPool(2);
```

It supports:

- one-time delayed tasks
- fixed-rate periodic tasks
- fixed-delay periodic tasks

It is generally preferred over `Timer` for concurrent scheduled execution.

43. Run a Task After a Delay

```
scheduler.schedule(  
    () -> System.out.println("Executed after 2 seconds"),  
    2,  
    TimeUnit.SECONDS  
);
```

The delay is relative to the time of submission.

44. `scheduleAtFixedRate()`

```
scheduler.scheduleAtFixedRate(  
    () -> System.out.println("Running"),  
    0,  
    2,  
    TimeUnit.SECONDS  
);
```

The intended start times are based on a fixed schedule:

```
initialDelay  
initialDelay + period  
initialDelay + 2 × period  
...
```

If one execution takes longer than the period, the next execution starts late.
Executions of the **same periodic task do not overlap with one another**.

Example:

```
Period = 2 seconds  
Task duration = 5 seconds  
  
Start 1: 0s  
End 1: 5s  
Start 2: approximately 5s, already behind schedule
```

Use fixed rate when the schedule should attempt to maintain a regular frequency.

45. `scheduleWithFixedDelay()`

```
scheduler.scheduleWithFixedDelay(  
    () -> System.out.println("Running"),
```

```
0,  
2,  
TimeUnit.SECONDS  
);
```

The delay is measured after the previous execution finishes.

```
Previous execution finishes  
↓  
Wait for fixed delay  
↓  
Start next execution
```

Example:

```
Task duration = 5 seconds  
Delay = 2 seconds  
  
Start 1: 0s  
End 1: 5s  
Start 2: 7s
```

Use fixed delay when the system should pause for a defined duration between executions.

Fixed Rate vs Fixed Delay

Feature	Fixed Rate	Fixed Delay
Scheduling basis	Planned start times	Previous completion time
Tries to maintain frequency	Yes	No
Waits after task finishes	Not necessarily	Yes
Same task overlaps with itself	No	No
Common use	Metrics collection, periodic polling with regular cadence	Cleanup or retry work needing a pause after completion

46. Exceptions in Periodic Tasks

If a periodic task throws an uncaught exception, its future executions are suppressed.

```
scheduler.scheduleAtFixedRate(() -> {
    try {
        performScheduledWork();
    } catch (Exception exception) {
        System.out.println(
            "Scheduled task failed: " +
            exception.getMessage()
        );
    }
}, 0, 10, TimeUnit.SECONDS);
```

Handle expected failures inside long-running periodic jobs when later executions must continue.

Rejection Policies

47. When Is a Task Rejected?

A task may be rejected when:

- the executor has been shut down, or
- the pool is at `maximumPoolSize` and its bounded queue is full

A `RejectedExecutionHandler` decides what should happen.

48. `AbortPolicy`

This is the default policy.

```
new ThreadPoolExecutor.AbortPolicy();
```

Behavior:

```
Reject task
↓
Throw RejectedExecutionException
```

It provides immediate and visible failure.

Suitable when silently losing work is unacceptable.

49. **CallerRunsPolicy**

```
new ThreadPoolExecutor.CallerRunsPolicy();
```

The rejected task runs in the thread that called `execute()`.

```
Executor overloaded
↓
Submitting thread executes task
↓
Producer becomes slower
```

This can create natural backpressure because the producer is temporarily occupied executing the task instead of submitting more work.

A task is discarded when the executor is already shut down.

50. **DiscardPolicy**

```
new ThreadPoolExecutor.DiscardPolicy();
```

The rejected task is silently dropped.

This can lead to unnoticed data loss and should only be used when dropping work is explicitly acceptable and monitored.

51. **DiscardOldestPolicy**

```
new ThreadPoolExecutor.DiscardOldestPolicy();
```

When possible, it removes the task at the head of the queue and retries submission of the new task.

```
Queue full
  ↓
Remove oldest queued task
  ↓
Retry new task
```

This policy may be useful when fresh information is more valuable than stale queued work, but it is dangerous for tasks that must all be processed.

52. Executor With a Rejection Policy

```
ThreadPoolExecutor executor =
    new ThreadPoolExecutor(
        2,
        4,
        30,
        TimeUnit.SECONDS,
        new ArrayBlockingQueue<>(10),
        new ThreadPoolExecutor.CallerRunsPolicy()
    );
```

This creates:

- two core workers
- at most four workers
- a queue of ten tasks
- thirty-second timeout for excess idle workers
- caller-runs behavior during overload

Important Design Principles

53. Separate Tasks From Threads

Write application logic as `Runnable` or `Callable` tasks.

Let the executor decide how worker threads are managed.

```
Business logic → task
Concurrency policy → executor configuration
```

This separation makes the code easier to test and change.

54. Do Not Assume More Threads Always Mean Better Performance

The useful number of threads depends on:

- available CPU cores
- whether tasks are CPU-bound or I/O-bound
- task duration
- blocking time
- memory limits
- downstream service capacity

CPU-Bound Tasks

Examples:

- image processing
- compression
- encryption
- numerical calculations

Too many threads usually add scheduling overhead without increasing available CPU.

I/O-Bound Tasks

Examples:

- database calls
- network calls
- file operations

More concurrency may help because some threads wait while others run. However, the pool must still respect database connection limits, remote-service limits and memory constraints.

55. Prefer Bounded Resources in Production

A production executor should normally have explicit limits.

Consider controlling:

- queue capacity
- maximum worker count
- submission rate
- task timeout
- rejection behavior
- downstream concurrency

An unbounded queue merely moves overload from thread creation to memory consumption.

56. Preserve Interruption

Do not silently ignore `InterruptedException`.

Incorrect:

```
try {
    Thread.sleep(1000);
} catch (InterruptedException exception) {
    // ignored
}
```

Preferred:

```
try {
    Thread.sleep(1000);
} catch (InterruptedException exception) {
    Thread.currentThread().interrupt();
    return;
}
```

Restoring the interrupt flag allows higher-level code to observe the cancellation request.

57. Name Worker Threads

Thread names make logs and production debugging easier.

```
AtomicInteger counter = new AtomicInteger();

ThreadFactory threadFactory = task -> {
    Thread thread = new Thread(task);
    thread.setName(
        "order-worker-" +
        counter.incrementAndGet()
    );
    return thread;
};
```

Use it with a custom executor:

```
ThreadPoolExecutor executor =
    new ThreadPoolExecutor(
        2,
        4,
        30,
        TimeUnit.SECONDS,
        new ArrayBlockingQueue<>(100),
        threadFactory,
        new ThreadPoolExecutor.CallerRunsPolicy()
    );
```

58. Monitor the Pool

Useful `ThreadPoolExecutor` metrics include:

```
executor.getPoolSize();
executor.getActiveCount();
executor.getQueue().size();
executor.getCompletedTaskCount();
executor.getTaskCount();
executor.getLargestPoolSize();
```

These values help answer:

- Is the queue continuously growing?
- Is the pool regularly reaching its maximum size?
- Are tasks completing fast enough?
- Is the executor frequently overloaded?

Complete Example

59. Controlled Order-Processing Executor

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.Future;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public class OrderProcessingDemo {

    public static void main(String[] args) {

        ThreadPoolExecutor executor =
            new ThreadPoolExecutor(
                2,
                4,
                30,
                TimeUnit.SECONDS,
                new ArrayBlockingQueue<>(10),
                new ThreadPoolExecutor.CallerRunsPolicy()
            );

        Future<String> future = executor.submit(() -> {
            System.out.println(
                "Processing order on " +
                Thread.currentThread().getName()
            );

            Thread.sleep(1000);

            return "ORDER_PROCESSED";
        });

        try {
            String result = future.get();
            System.out.println(result);
        }
    }
}
```

```

    } catch (InterruptedException exception) {
        Thread.currentThread().interrupt();

    } catch (ExecutionException exception) {
        System.out.println(
            "Order failed: " +
            exception.getCause().getMessage()
        );
    } finally {
        executor.shutdown();
    }
}

```

This example demonstrates:

- explicit pool configuration
- bounded task queue
- overload handling
- a value-returning task
- result retrieval through `Future`
- exception handling
- proper executor shutdown

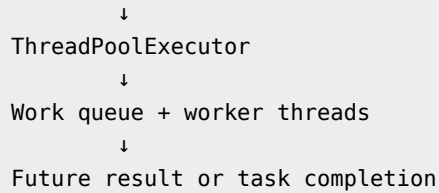
Quick Revision

60. Core Concepts

```

Runnable / Callable
    ↓
ExecutorService

```



61. Key Differences

Topic	Main Idea
Raw thread	Low-level thread creation and management
Executor	Abstraction for executing a <code>Runnable</code>
ExecutorService	Executor with lifecycle, results and bulk operations
ThreadPoolExecutor	Configurable thread-pool implementation
<code>execute()</code>	Submit a <code>Runnable</code> without receiving a <code>Future</code>
<code>submit()</code>	Submit a task and receive a <code>Future</code>
<code>Runnable</code>	No returned value; no declared checked exception
<code>Callable</code>	Returns a value and may throw checked exceptions
<code>Future</code>	Tracks completion, result and cancellation
<code>shutdown()</code>	Stop new submissions and finish accepted tasks
<code>shutdownNow()</code>	Attempt interruption and return waiting tasks
<code>invokeAll()</code>	Wait for all tasks
<code>invokeAny()</code>	Return one successful result
Fixed pool	Fixed workers with an unbounded queue
Cached pool	On-demand workers with direct handoff
Single-thread executor	Sequential execution using one worker
Scheduled executor	Delayed and periodic execution

Final Takeaway

The Executor Framework is not simply a shorter way to create threads.

It provides a complete task-execution system with:

- thread reuse
- controlled concurrency
- task queuing
- asynchronous results
- cancellation
- lifecycle management
- scheduling
- overload handling

The most important mental shift is:

Do not think, "I need to create a thread."

Think, "I need to submit a task to an execution policy."